# Important Themes in Programming

Appendix B

© 2017 Pearson Education, Hoboken, NJ.  All rights reserved

# Important Themes in Programming

- Strive for **modularity** in all phases of the problem-solving process, beginning with the initial design of a solution
- Develop more **smaller modules** versus fewer larger modules
  - Helps with debugging
  - Improves supportability and readability
  - Reduces redundant code
- Hide the exact representation of these data members from modules that use the object by making all of the **data members private**; the object should provide
  - a simple method—called an accessor (getter) —that returns the value of the data member and
  - another method—called a mutator (setter)—that sets the value of the data member
- Proper Use of Reference Arguments
  - **Value** parameters, (do not write & after the parameter's data type), pass values into the method, where the value of each argument is copied into the parameter; any change that the method makes to these parameters is not reflected in the actual arguments back in the calling program.
  - **Reference** parameter becomes an alias (another name) for the corresponding argument, changes to the parameter within the method also occur in the argument in the calling module.
  - When copying the argument is expensive, use pass-by-constant reference instead of pass-by-value.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Important Themes in Programming

- To reduce **coupling** among modules, you should restrict the calls that a method can make. A method should call only other methods:
  - Defined within the same class
  - Of argument objects
  - Of objects created within the method
  - Of objects contained within the same class as data members
- Do **NOT** use **global variables**.
- Check for errors in both its input and its logic and attempt to behave gracefully and consistently when it encounters them. A method should check for certain types of errors, such as **invalid input or argument values.**
- Appropriate action in the **face of an error** can range from ignoring erroneous data and continuing execution to terminating the program. A common technique is for a method to return a boolean value to the calling module to indicate that it has encountered an error.
- The method leaves error handling to the calling module. In general, methods should either **return a value** or **throw an exception** instead of displaying a message when an error occurs. (Write to an error log file.)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Important Themes in Programming

- Choose identifiers that **describe** their purpose—that is, that are self-documenting. Distinguish between keywords, such as int, and user-defined identifiers.
- Use a good **indentation** style to enhance the readability of a program. The layout of a program should make it easy for a reader to identify the program's modules.
- Use a **blank line** between method or function definitions.
- Within these definitions, you should **indent** individual blocks of code. These blocks are generally—but are not limited to—the actions performed within a control structure, such as a while loop or an if statement.
- Named constants make a program easier to modify, e.g.,

```
const int NUMBER_OF_MAJORS = 202;
```

- typedef statements make a program easier to modify; if your program performs floating point computations, and you need greater precision than float (change float to double)
  - typedef float RealType; // original statement
  - typedef long double RealType; // update to this statement in one spot

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Important Themes in Programming

- Two kinds of testing can be applied to a system:
  - Open-box testing (**white-box** or glass-box testing) - design tests knowing how the modules work.
    - choose a method's actual arguments in a way that exercises all lines of code in the method.
    - make sure that all branches of if and switch statements are executed.
  - Closed-box testing (**black-box** or functional testing) - you know the name of the method being tested, as well as its argument list, its return type, and its operation contract.
    - develop tests across a range of valid input, and then check the output against hand-calculated results.
    - Choose arguments outside of their valid range tests how the module handles incorrect input.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Important Themes in Programming

- This book uses a **javadoc-style** comment (originated with the Java™ programming language).
- These comments start with /** and end with */ and contain tags that classify different parts of your documentation.
- Several utility programs are available to read these comments and generate HTML-based documentation.
- The advantage of using javadoc-style comments—and a documentation system, like **doxygen**—is that you can edit the documentation for your source code as you make changes to that code.
- The documentation lives directly in the source code—as javadoc-style comments—making it very easy and convenient to keep your documentation **up to date**.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# Important Themes in Programming

Essential features of program documentation

1. An **initial comment at the top of each source code file** that includes:
   a) File name (@file)
   b) Statement of purpose
   c) Author (@author)
   d) Date (@date)
   e) Optional file version number (@version)
2. Initial **comments for each class** that includes:
   a) Name of class and its header file (@class)
   b) Statement of purpose
3. Initial **comments for each method or function** that includes:
   a) Statement of purpose
   b) Description of each argument in the argument list (@param)
   c) Preconditions (@pre)
   d) Postconditions (@post)
   e) Exceptions thrown (@throw)
   f) Return value (@return)
4. Standard C++ comments in the body of each method or function to explain **important features** or subtle logic

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

# End

Appendix B

© 2017 Pearson Education, Hoboken, NJ.  All rights reserved